**Laborator 1**

# Java Classes

Sumar

Clasa, constructor
Instanta, cuvant cheie *this*
Interfata, clasa abstracta
Mostenire, cuvantul cheie *super*
Overload/Override
Agregare vs compunere

## 1.1.1 Clasa

- Se foloseste cuvantul cheie "**class**"

```
//public accessible class, defined in a file with same name.
public class PhoneAgenda {
    ....
}

//package protected, final class
final class PhoneAgendaB {

}

//public accessible class
public class PhoneAgendaC {
    private String owner;
    private InnerAgenda inner;

    private void init() {
        inner = new InnerAgendaC();
        inner.innerField = "A";
        System.out.println(inner.innerField);
    }

    public static void main(String args[]) {
        PhoneAgenda aC = new PhoneAgendaC();
        a.init();
        a.inner.innerField = "B"; //method main is in same class,
                                  //thus we can access innerField.
        System.out.println(a.inner.innerField);
    }

    private class InnerAgenda {
        private String innerField;
    }
}
```

## 1.1.2 Constructori

Fiecare clasa poate avea 1 sau mai multi constructori. Daca nu declaram niciun constructor, compilatorul va adauga constructorul implicit (default constructor), acel constructor fara niciun parametru.

```
public class Student {

    private String name;      // field name
    public int averageGrade;  // field averageGrade

    // (1) constructor without parameters
    public Student() {
        name = "Unknown";
        averageGrade = 5;
    }

    //(2) constructor with two parameters; used to set the name and the grade
    public Student(String n, int avg) {
        name = n;
        averageGrade = avg;
    }

    // (3) constructor with one parameter; used to set only the name
    public Student(String n) {
        this(n, 5); // call the second constructor (2)
    }

    // (4) setter for the field 'name'
    public void setName(String n) {
        name = n;
    }

    // (5) getter for the field 'name'
    public String getName() {
        return name;
    }
}
```

Crearea unui obiect Student se face obligatoriu prin apel la unul din cei 3 constructori de mai sus:

```
Student st;
st = new Student();           // first constructor call (1)
st = new Student("Gigel", 6); // second constructor call (2)
st = new Student("Gigel");    // third constructor call (3)
Student s = new Student();    // compile error. No arg constructor not defined.
```

Copy constructor:

```
// copy constructor
public Student(Student student) {
    // name este camp privat, noi il putem accesa direct (student.name)
    // deoarece ne aflam in interiorul clasei
    this.name = student.name;
```

```
        this.averageGrade = student.averageGrade;
    }
```

### 1.1.3 Cuvantul cheie *this*

Cuvantul cheie `this` se refera la instanța curentă a clasei și poate fi folosit de metodele, care nu sunt statice, ale unei clase pentru a referi obiectul current (instanta curenta).
In general, cuvântul cheie this este utilizat pentru:

- a se face diferența între câmpuri ale obiectului curent și argumente care au același nume
- a pasa ca argument unei metode o referință către obiectul curent (vezi liniile (*) si (**) din exemplul următor)
- a facilita apelarea constructorilor din alți constructori, evitându-se astfel replicarea unor bucăți de cod (vezi exemplul de la constructori)

```
class Group {

    private int numberStudents;
    private Student[] students;

    public Group () {
        numberStudents = 0;
        students = new Student[10];
    }

    public boolean addStudent(String name, int grade) {
        if (numberStudents < students.length) {
            students[numberStudents++] = new Student(this, name, grade); // (*)
            return true;
        }

        return false;
    }
}
class Student {

    private String name;
    private int averageGrade;
    private Group group;

    public Student(Group group, String name, int averageGrade) {
        this.group        = group;            // (**)
        this.name         = name;             // (**)
        this.averageGrade = averageGrade;     // (**)
    }
}
```

### 1.1.4 Interfata

- Folosirea cuvantului cheie “**interface**”, **“implements”**
- Metode fara corp
- Incepand cu java 1.8 se pot adauga
- “default methods” care au corp executabil. Poate fi suprascris in clasa mostenitoare
- metode statice

**Laborator 1**

- @FunctionalInterface – interfata are doar o singura metoda. Utilizabila in expresii lambda

```
public interface IShape {

    int numberOfDots();

    int[] lengths();

    double calcArea();
}

public class Triangle implements IShape {
    int[] lengths;

    public Triangle(int[] lengths) {
        this.lengths = lengths;
    }

    @Override
    public int numberOfDots() {
        return 3;
    }

    @Override
    public int[] lengths() {
        return lengths;
    }

    @Override
    public double calcArea() {
        return 0;
    }
}

public class Rectangle implements IShape {
    int[] lengths;

    public Rectangle(int[] lengths) {
        this.lengths = lengths;
    }

    @Override
    public int numberOfDots() {
        return 4;
    }

    @Override
    public int[] lengths() {
        return lengths;
    }

    @Override
    public double calcArea() {
        if (lengths != null && lengths.length == 2) {
            return lengths[0] * lengths[1];
        }
        return -1;
```

```
    }
}
```

### 1.1.5 Clasa abstracta

Este o clasa care poate contine sau nu metode abstracte. Ea nu poate fi instantiata. Ea poate fi mostenita.

O metoda abstracta este o metoda fara corp.

Metode abstracte exista doar intr-o clasa abstracta.

```
public abstract class AbstractShape {
    private int count = 0;

    public abstract String getName();

    protected abstract int getWidth();

    protected abstract int getLength();

    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append("[Shape] ").append(getName());
        sb.append(" width = ").append(getWidth());
        sb.append(" length = ").append(getLength());
        sb.append(" count = " + (count++));
        return sb.toString();
    }
}

class RectangleShape extends AbstractShape {
    private int w;
    private int h;

    public RectangleShape(int w, int h) {
        this.w = w;
        this.h = h;
    }

    @Override
    public String getName() {
        return "RectangleShape";
    }

    @Override
    protected int getWidth() {
        return w;
    }

    @Override
    protected int getLength() {
        return h;
    }
}
```

## 1.1.6 Mostenire

- Cuvantul cheie extends, implements
- *extends*: permite unei clase sa mosteneasca o singura clasa concrete sau abstracta. Permite Unei interfete sa mosteneasca  mai multe interfete
- *implements*: permite unei clase sa implementeze mai multe interfete

Numita si derivare, mostenirea este un mecanism de refolosire a codului specific limbajelor orientate obiect si reprezintă posibilitatea de a defini o clasa care extinde o alta clasă deja existenta. Ideea de baza este de a prelua funcționalitatea existenta într-o clasa si de a adăuga una noua sau de a o modela pe cea existenta.

```
public interface IListener {
    void notify(String data);
}


public class Rectangle implements IShape, Listener {

    int[] lengths;

    public Rectangle(int[] lengths) {
        this.lengths = lengths;
    }

    @Override
    public int numberOfDots() {
        return 4;
    }

    @Override
    public int[] lengths() {
        return lengths;
    }

    @Override
    public double calcArea() {
        if (lengths != null && lengths.length == 2) {
            return lengths[0] * lengths[1];
        }
        return -1;
    }

    @Override
    public void notify(String data) {
        System.out.println("I have been notified with: "+data);
    }
}

public interface IListener {

    void notify(String data);
}
```

```
public class Rectangle implements IShape, Listener {

    int[] lengths;

    public Rectangle(int[] lengths) {
        this.lengths = lengths;
    }

    @Override
    public int numberOfDots() {
        return 4;
    }

    @Override
    public int[] lengths() {
        return lengths;
    }

    @Override
    public double calcArea() {
        if (lengths != null && lengths.length == 2) {
            return lengths[0] * lengths[1];
        }
        return -1;
    }

    @Override
    public void notify(String data) {
        System.out.println("I have been notified with: "+data);
    }
}


import java.util.Observer;
import java.util.Observable;
public class EnhancedRectangle extends Rectangle {

    private Observer observer;

    public EnhancedRectangle(int[] lengths) {
        super(lengths);
    }

    public void setObserver(Observer observer) {
        this.observer = observer;
    }

    @Override
    public void notify(String data) {
        if (observer != null) {
            Observable observable = new Observable();
            observable.addObserver(observer);
            observable.notifyObservers(data);
        }
    }
}
```

### 1.1.7 Cuvantul cheie super

Este utilizat pentru a accesa campuri, metode sau constructor in clasa parinte.

```
public class Car {
    private String producer;
    public Car(String producer) {
        this.producer = producer;
    }
    public String getProducer() {
        return producer;
    }
}

class Dacia extends Car {
    private String owner;
    public Dacia(String owner) {
        super("Dacia Mioveni SRL"); // *
        this.owner = owner;
    }
    public String getOwner() {
        return owner;
    }
}
```

### 1.1.8 Overload/override (supraincarcare/suprascriere)

Suprascrierea (overriding) presupune înlocuirea functionalitatii din clasa/clasele parinte pentru instanta curenta.

Supraincarcarea (overloading) presupune furnizarea de funcționalitate aditionala, fie pentru metodele din clasa curenta, fie pentru clasa/clasele parinte.

```
public class Car {
    public void print() {
        System.out.println("Car");
    }

    public void init() {
        System.out.println("Car");
    }

    public void addGasoline() {
        // do something
    }
}

class Dacia extends Car {
    // suprascriere
    public void print() {
        System.out.println("Dacia");
    }
```

```
     // suprascriere
    public void init() {
        System.out.println("Dacia");
    }

    // suprascriere
    public void addGasoline() {
        // do something
    }

    // supraincarcare
    public void addGasoline(Integer gallons) {
        // do something
    }
}
Car a = new Car();
Car b = new Dacia();
Dacia c = new Dacia();
Car d = null;

a.print(); // afiseaza Car
b.print(); // afiseaza Dacia
c.print(); // afiseaza Dacia
d.print(); // arunca NullPointerException
```

### 1.1.9 Agregare, compunere

Agregarea și compunerea se referă la prezența unei referințe pentru un obiect într-o altă clasă. Acea clasă practic va refolosi codul din clasa corespunzătoare obiectului.

Agregarea (aggregation) - obiectul-container poate exista și în absența obiectelor agregate, de aceea este considerată o asociere slabă (weak association). În exemplul de mai jos, un raft de bibliotecă poate exista și fără cărți.

Compunerea (composition) - este o agregare puternică (strong), indicând că existența unui obiect este dependentă de un alt obiect. La dispariția obiectelor conținute prin compunere, existența obiectului container încetează. În exemplul de mai jos, o carte nu poate exista fără pagini.

```
class Page {
    private String content;
    public int numberOfPages;

    public Page(String content, int numberOfPages) {
       this.content       = content;
       this.numberOfPages  = numberOfPages;
    }
}

class Book {
    private String title;           // Compunere
    private Page[] pages;           // Compunere
    private LibraryRow libraryRow = null;   // Agregare

    public Book(int size, String title, LibraryRow libraryRow) {
```

```
        this.libraryRow = libraryRow;
        this.title = title;

        pages = new Page[size];

        for (int i = 0; i < size; i++) {
           pages[i] = new Page("Page " + i, i);
        }
    }
}

class LibraryRow {
    private String rowName = null;      // Agregare

    public LibraryRow(String rowName) {
        this.rowName = rowName;
    }
}
```

**Laborator 1**

## 1.2 Proiect IntelliJ Idea + GitHub

### 1.2.1 Cont GitHub

Daca nu aveti deja, realizati cont GitHub astfel: https://github.com/ > SignUp > ulbs email > SignUp > skip personalization

(Folositi emailul din cadrul universitatii ULBS. Memorati userul si parola)

### 1.2.2 GitHub in Intellij Idea

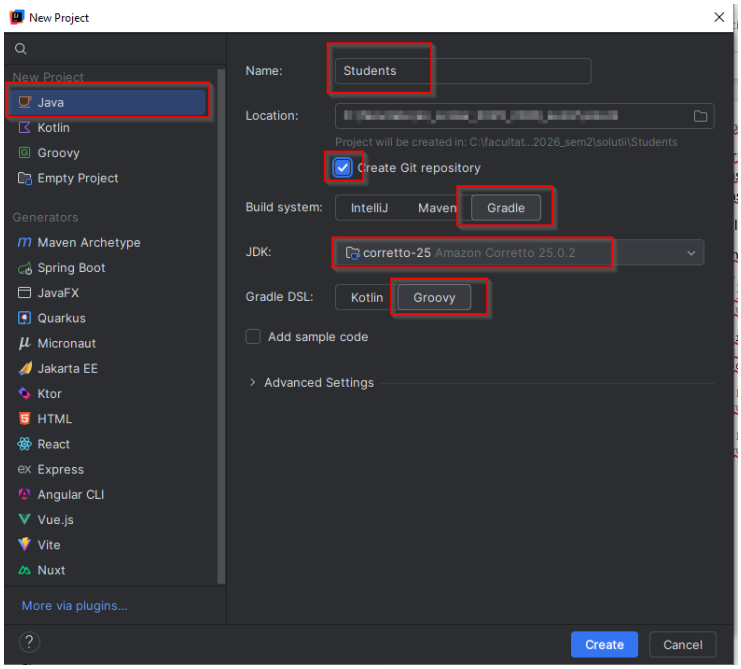
IntelliJ Idea: File > Settings > Version Control > GitHub > Add Account

## 1.3 Hands-on

### 1.3.1 Students

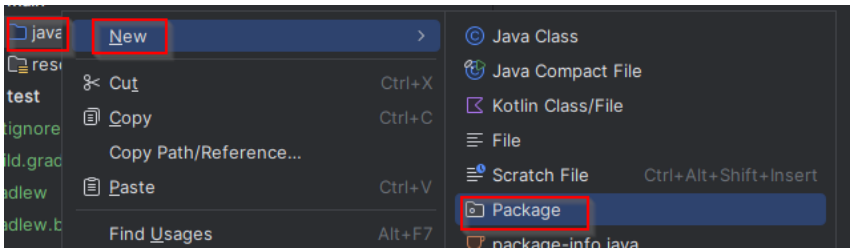
Creati un nou proiect numit 'Students'.

IntelliJ Idea > New Project
Language: Java,
Build System: Gradle




Vom folosi acest proiect si in laboratoarele urmatoare.

Creati pachetul *ro.ulbs.proiectaresoftware.students* .



**Laborator 1**

In acest subpachet creati clasa Student. Apoi declarati campurile: *numarMatricol*, *prenume*, *nume*, *formațieDeStudiu.* Tipurile de date sunt int, String, String si respectiv String.

Acum generati (click dreapta > Generate) constructor cu toate aceste campuri, dar si metode getter.

Generati metoda **toString().**

## Application

Adaugati o noua clasa, numita *Application* in acelasi pachet, care sa contina metoda *public static void main().*

In aceasta metoda instantiati 5 studenti si apoi afisati:

```
Student s1 = new Student(112, "Ioan", "Popa", "TI21/1");
Student s2 = new Student(112, "Maria", "Oprea", "TI21/1");
Student s3 = new Student(120, "Alis",  "Popa", "TI21/2");
Student s4 = new Student(122, "Mihai", "Vecerdea", "TI22/1");
Student s5 = new Student(122, "Eugen", "Uritescu", "TI22/2");
```

## Formatare output

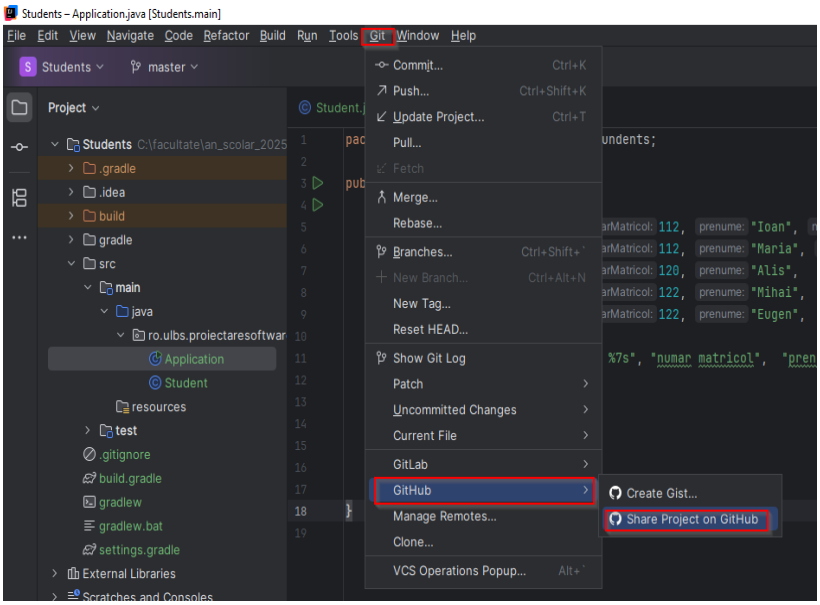
Modificati metoda toString() astfel incat sa aveti formatare pe coloane. Folositi String.format().

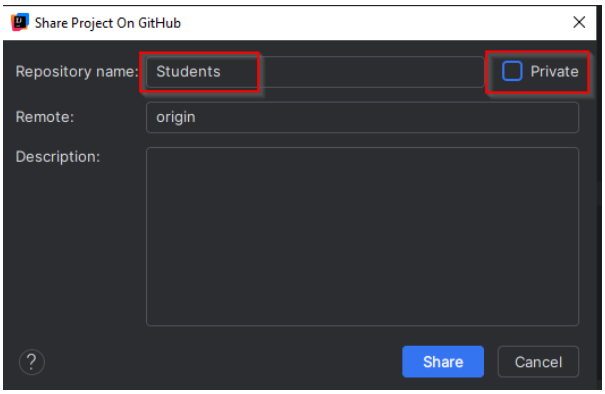
```
Task :ro.ulbs.proiectaresoftware.stundents.Application.main()
numar matricol         prenume nnume formatieDeStudiu
           112             Ioan Popa  TI21/1
           112            Maria Oprea  TI21/1
           120              Alis Popa  TI21/2
           122          Mihai Vecerdea  TI22/1
           122          Eugen Uritescu  TI22/2
```

## 1.3.2 Adaugati proiectul in GitHub

Publish to GitHub:

IntelliJ: Git > GitHub > Share Project on GitHub




Daca ati facut repository-ul privat, fie

a) convertiti proiectul facandu-l public. Web: GitHub repo > Settings > General > scroll to 'Danger Zone' > Change repository visibility

sau ma adaugati collaborator:

b) Web: GitHub > Repositories > lab1 > Collaborators > Add people

Adaugati pe GoogleClassroom linkul la acest repo pentru ca eu sa pot evalua commiturile.

**Laborator 1**

1.3 Creati un nou proiect, LaboratoareSoftware.

a) creati pachetul lab1 (sau ro.ulbs.proiectaresoftware.lab1). In acest pachet adaugati clasele urmatoare dar si o clasa care sa contina metoda main (clasa se poate numi Application sau chiar Main)

b) Generati literele alfabetului intr-o instanta de tip String. Nu enumerate literele, ci calculate folosind instructiunea for(…)

Afisati pe primul rand aceasta instanta cu caractere mici, iar pe al doilea cu majuscule. (folositi toLowerCase() si toUpperCase() ale clasei String).

c) Pe baza instantei de la pasii anteriori, realizati un array de string-uri astfel: fiecare element in array contine sirul de caractere ce incepe cu o vocala si se opreste pe consoana dinaintea urmatoarei vocale. Afisati acest array.

Ex: Plecand de la instanta "abcdefghijklkmnop…"

```
array[0] = "abcd";
array[1] = "efgh";
array[2] = "ij..";
...
```

Publicati si acest proiect in GitHub, si adaugati linkul repo-ului pe classroom.

## 1.5 Referinte aditionale

How to design Classes and Interfaces
https://www.javacodegeeks.com/wp-content/uploads/2015/10/Advanced-java.pdf

UML Diagrams
https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/

Association vs Aggregation vs Composition
https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-aggregation-vs-composition/

Multiple Inheritance
https://www.journaldev.com/1775/multiple-inheritance-in-java

Advanced Java
https://www.javacodegeeks.com/wp-content/uploads/2015/10/Advanced-java.pdf